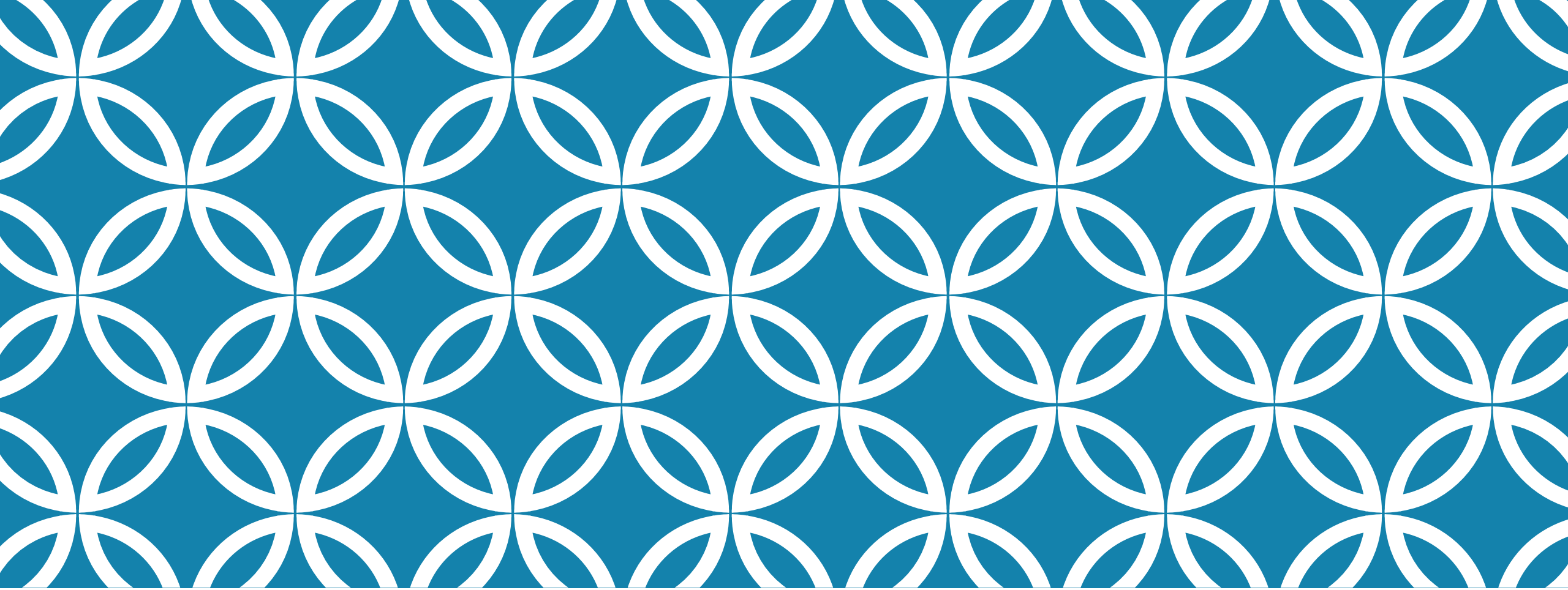


STATIC PROGRAM ANALYSIS

Lecture 1 - Intro

STATIC PROGRAM ANALYSIS

- A technique that
  - Computes information about a program without executing the program (static vs. dynamic)
  - Reasons about all possible program executions
  - Produces sound results, i.e., no false negatives (never misses), but could give false positives (gets extra things in)
    - Does a program have a division by zero fault?
    - No – no concrete executions have a division by zero
    - Yes – maybe one concrete execution has a division by zero

TYPES OF STATIC ANALYSIS

- There are many “flavors” of static analysis and static analyzers
  - Used in different contexts, i.e., applications (next slide)
  - Compute different information about programs
  - Use different program representations
  - Improve precision of other analysis
    1. Find what variables point to which objects, assuming all branches are feasible
    2. Using this info, another analysis can identify infeasible branches
    3. Go to step 1
- The goal of the class is to sort those ”flavors” out and understand how they relate to each other

APPLICATIONS OF STATIC PROGRAM ANALYSIS

Compiler optimization
- Register allocation
- Constant identification

Program verification
- Runtime errors
- Assertion violation

Refactoring
- Renaming variable
- Extracting class/method

Code styles checks
- Naming conventions
- Visibility modifiers

Code development (IDE)
- Unused/uninitialized variables
- Method look-ups
- Call hierarchy

Testing
- Coverage elements
- Test input generation

Other?

STATIC ANALYSIS DEVELOPMENT PROCESS

We will use the following process to learn various static analysis

- task to accomplish: detect a dead code; clear unused registers; check that no null pointer exception present
- state it as a decision problem using computed information: what is the value of Boolean expression in conditional statements; is a variable used later in computation; what objects a reference variable points to
- instantiate SA: reasons about a program and computes required information
- examine results: use the computed data to decide the problem (yes or no, i.e., a decision problem)

STEP 1: DEFINING A PROBLEM

- What tasks software engineers need to accomplish?
- High-level definitions, which more likely need refinements (e.g., my code is bug-free)
- What type of bugs?
  - division by zero
  - null pointer dereference
  - assertion should hold
- Decomposing into a collection of simpler problems
  - Modern/industrial static analyzers run as a cooperative collection of analyses
  - Analysis can be composed in many different ways
- Problems that seem related might require distinct approaches
  - Two code fragments are equivalent
  - Two code fragments are different

STEP 2: STATING A PROBLEM AS A DECISION PROBLEM

- An essential skill – one of the objectives of this class
- Express a software engineering problem as a decision problems over computed information
  - P1: Whether a division statement throws a division by zero exception.
  - DP1: Can a value of a divisor be zero?
  - Ideally: DP1: yes → P1: yes, DP1: no → P1: no, and vice versa, e.g., P1 ↔ DP1
  - Halting problem → Rice's theorem: nontrivial property about program behaviors → undecidable problem
  - Strategy: DP1 over-approximates P1
  - Commonly:
    - DP1: yes → P1: yes/no; DP1: no → P1: no
    - Conversely, P1: yes → DP1: yes, P1: no → DP1: yes/no
- Identify what type of data is required to answer that decision problem
  - What all possible values that the divisor can take on?
- Determine how to compute that data
  - Consider all possible executions of a program
  - Accumulate those values as the program gets interpreted

STEP 3: INSTANTIATE A STATIC ANALYSIS

- There are many parametrized frameworks
  - Data-flow analysis framework (how to propagate data through program) (Kincaid'73)
  - Abstract Interpretation framework (data representation, usually an abstraction of concrete data) (Cousot'77)
- Understand what parameters to choose to
  - Ensure correctness, i.e., ensuring that the correct data is computed
  - Increase precision, i.e., low level of false positives
  - Maintain efficiency, i.e., time to run an analysis
  - Usually precision and efficiency are trade-offs
- We will look at many examples of parameter choices and trade-offs
  - WALA static analyzer from IBM

STEP 4: EXAMINE RESULTS

- Check for the computed values
- Answer the decision problem
- Determine false positives
- Examine whether some parameters should be
  - Modified, i.e., changed completely
  - Tuned, i.e., increase/decrease precision

PROGRAM VIEWS

- How do we view a program?
- What is a program?

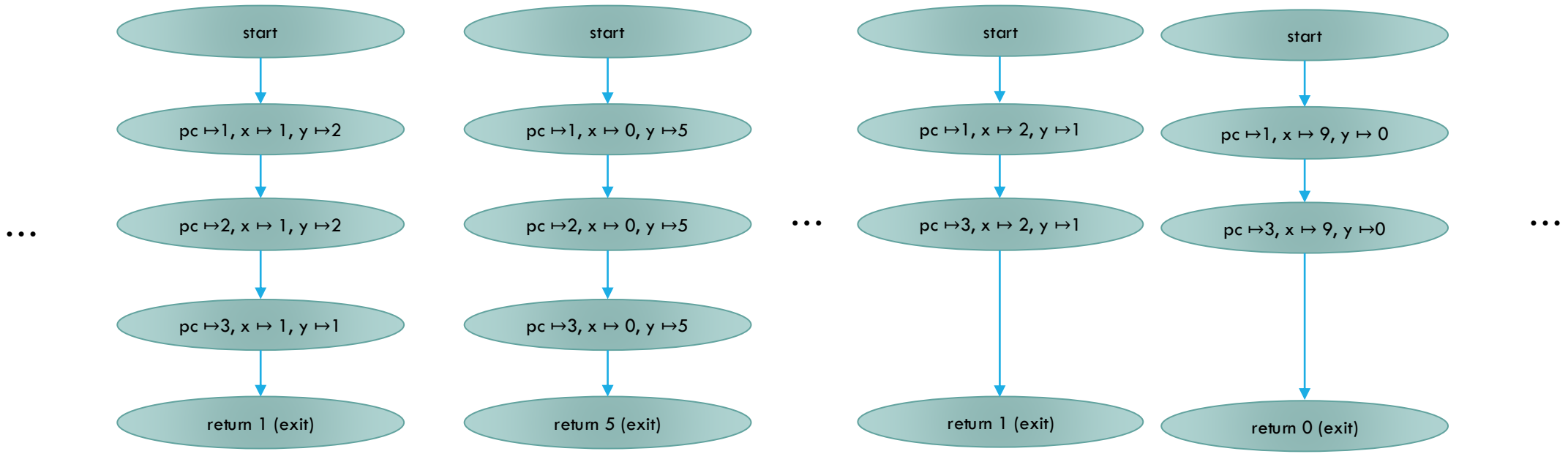
TWO EXTREME VIEWS

Text

Many views in between

Set of concrete execution traces

```
int min(int x, int y){
1: if( x < y){
2:  y = x;
}
3: return y;
}
```



Trace a sequence of states
State a map from variable to their values + program counter

PROGRAM ABSTRACTIONS

- Type of different views/abstractions of a program
  - Text: sequence of symbols
    - Find a pattern
    - Helpful in many simple tasks: does the code have an @author tag?
  - Code: an abstract syntax tree (AST)
    - Attaches meanings to tokens (nodes) and creates relations between them (labeled edges)
    - Concrete syntax tree also includes semicolons, curly braces and other concrete syntax-related nodes.
  - Code flow: a control flow graph (CFG)
    - Statements (usually in 3-address code representation) and flow dependencies between them
      - Sequential – a block of statements
      - Choice – conditional statements
  - Code flow: a data flow graph (DFG)
    - Data dependencies between computed values
    - Used in code parallelization to compute independent data-flow simultaneously

NEXT TIME

- Look at some simple problems
- Treating program like a text
- Starting with AST